Enhancing Metaheuristics for Pistachio Supply Chain Optimization: A Variable Neighborhood Search Algorithm with Problem-Specific Operators

Raissa Gonçalves Diniz[a], André Costa Batista[a,b]

[a] *Operations Research and Complex Systems Laboratory, Universidade Federal de Minas Gerais, Brazil*
[b] *Department of Electrical Engineering, Universidade Federal de Minas Gerais, Av. Antônio Carlos 6627, Belo Horizonte, 31270-010, Minas Gerais, Brazil*

Abstract

Efficient supply chain management is critical for high-value agricultural products like pistachios. However, while mixed-integer linear programming models exist for this domain, the literature lacks efficient heuristic approaches with well-defined solution methodologies. To address this gap, we present an enhanced Variable Neighborhood Search (VNS) approach for pistachio supply chain optimization. Our method introduces an integer-based priority encoding scheme coupled with a formally defined decoding algorithm that ensures solution feasibility. Building on this representation, we developed domain-specific neighborhood structures that exploit the problem's constraint structure to improve search convergence beyond classical operators. Through comprehensive benchmarking against Genetic Algorithms (GA), Iterated Local Search (ILS), and an exact solver, we demonstrate that the proposed VNS outperforms other metaheuristics on medium and large-scale instances, achieving near-optimal solutions in scenarios where exact methods become computationally intractable.

Keywords: Supply Chain Optimization, Metaheuristics, Variable Neighborhood Search, Sustainable Logistics

Email addresses: raissagd@ufmg.br (Raissa Gonçalves Diniz), andre-costa@ufmg.br (André Costa Batista)

1. Introduction

The growing role of agriculture in the economic development of many countries highlights the importance of integrating this sector with other industries to stimulate overall economic growth. Agriculture not only supports food security but also acts as a major contributor to exports, generating significant income and strengthening other sectors. Among various agricultural products, pistachios stand out due to their high economic and nutritional value. Major producers such as the US, Turkey, and Iran dominate global pistachio production, remaining the leading producers worldwide in 2022 and 2023 (Statista, 2023). As a result, the strategic importance of pistachios in international trade continues to grow.

With this increasing global demand for pistachios, the need for efficient supply chain networks becomes more critical. Proper supply chain management is vital for maintaining product quality, minimizing environmental impacts, and ensuring the smooth flow of goods from production to consumers. A key challenge in pistachio production is the management of its by-products, such as hulls and shells, which, while free of heavy metals or pathogens, can degrade soil quality and contribute to environmental pollution if not properly handled (Doula et al., 2014; Bartzas and Komnitsas, 2017). Furthermore, improper disposal of these materials may lead to the growth of molds that increase the risk of aflatoxin contamination, posing serious health risks such as liver cancer (Hokmabadi, 2018; Dronavalli and Kang, 2019). Therefore, eco-friendly practices for reusing agricultural waste are essential to mitigate both environmental and health hazards.

Moreover, inefficiencies in supply chain networks for food and agricultural products, including pistachios, can have significant implications for product quality and trade (Casper and Sundin, 2018). Addressing these environmental and logistical concerns is crucial to sustaining both the economic and ecological viability of the pistachio industry. In the broader context of sustainable supply chains, several studies have focused on optimizing resource use and minimizing waste through recovery and recycling practices. For example, sustainable supply chain models designed for agriculture and other industries have shown significant environmental benefits by reducing waste and reusing by-products for processes like biogas and compost production (Cheraghalipour and Roghanian, 2022). Such approaches have been successfully applied in sectors such as manufacturing and waste recovery, where efficient resource management and cost reduction have been key drivers (De-

vika et al., 2014; Zohal and Soleimani, 2016). These strategies underline the importance of sustainability in modern supply chains and the potential for agricultural industries to implement similar measures.

In related research, Rajabi-Kafshgar et al. (2023) focused on optimizing the pistachio supply chain using various algorithms, primarily population-based approaches, such as the Genetic Algorithm (GA) (Fathollahi-Fard and Hajiaghaei-Keshteli, 2018; Xie et al., 2022), along with several metaphor-based metaheuristics including the Imperialist Competitive Algorithm and Social Engineering Optimizer. However, one aspect that could be improved in their methodology is the level of detail provided for reproducibility. Specifically, the authors employed a real-valued encoding scheme without detailing the decoding procedure required to transform these values into a feasible transport tree. This omission hinders reproducibility and obscures the mechanism by which constraints are handled. Additionally, recent critical analyses (Campelo and Aranha, 2023; Aranha et al., 2022) have demonstrated that many metaphor-based metaheuristics offer limited algorithmic innovation beyond terminological variations, often performing comparably to (or worse than) classical, well-established methods. Given these findings, this work adopts a more transparent approach by employing the Variable Neighborhood Search (VNS), a well-understood classical metaheuristic with clear operational principles, extensive theoretical foundation, and proven effectiveness across diverse optimization problems.

Addressing these gaps, this research focuses on designing an optimal supply chain network for pistachios by leveraging the Variable Neighborhood Search (VNS) algorithm (Gendreau and Potvin, 2010). While we adopt the same mixed-integer linear programming (MILP) model formulation from Rajabi-Kafshgar et al. (2023), we propose a robust overhaul of the resolution methodology. We introduce a transparent integer-based priority encoding and provide a step-by-step description of the decoding algorithm, ensuring reproducibility. Furthermore, we develop problem-specific neighborhood operators designed to exploit the particular structure of the chain. These specialized operators are contrasted with classical moves to demonstrate their efficacy in exploring the solution space.

To validate the proposed approach, we conduct a comprehensive comparative analysis against a Genetic Algorithm (GA), an Iterated Local Search (ILS), and an exact Branch-and-Cut method (using the Gurobi solver). Our findings demonstrate that the proposed VNS with specialized operators consistently outperforms the other metaheuristics in terms of solution quality.

On top of that, we show that for large-scale instances, where exact methods become infeasible due to memory limitations and excessive runtime, the proposed heuristic offers a vital trade-off, providing high-quality feasible solutions efficiently.

The remainder of this paper is organized as follows. Section 2 positions the present study within the literature on agri-food network design, priority-based encodings, and structure-aware neighborhoods. Section 3 presents the problem definition and the mathematical formulation. Section 4 details the heuristic approach, introducing the integer encoding, the specific decoding steps, and the domain-specific neighborhood structures. Section 5 reports the computational experiments, offering a statistical comparison between VNS, GA, ILS, and the exact solver across varying instance sizes. Finally, Section 6 summarizes the key findings and suggests future research directions.

2. Related Work

[TO BE WRITTEN] *Preamble stating the three bodies of literature this work draws on and the specific gap it addresses.*

2.1. Agri-food Supply Chain Network Design

[TO BE WRITTEN] *Must establish what distinguishes the pistachio setting from generic multi-echelon network design. The key structural differentiator is the joint production at processing centers: a single input stream splits into open-shell pistachios and raw kernels in fixed conversion proportions, so satisfying one downstream demand mechanically creates surplus of the other output. This is what the reconciliation step of the decoder (Algorithms 1–2) exists to handle, and it is currently underexploited as a contribution. Also covers by-product valorization and the aflatoxin/waste-handling motivation.*

2.2. Priority-Based Encodings for Network Design

[TO BE WRITTEN] *Must trace the lineage of the representation and state plainly that it is adopted, not invented: Gen and Cheng, Gen et al. (2006), random-key representations, and their use in multi-echelon logistics. Must then justify the specific design choice made here (integer permutation over random keys) on genotype-to-phenotype grounds, and note that the encoding is linear in the number of facilities, since one priority is assigned per node rather than per arc. Addresses Reviewer 1 comment 1 and Reviewer 2 comment 3.*

2.3. Structure-Aware Neighborhoods in Metaheuristics

[TO BE WRITTEN] *Must survey phenotype-aware, solution-dependent, and facility-activation neighborhoods, and establish explicitly how the four operators proposed here differ from that prior art. Reviewer 4 comment 1 states that this distinction is not adequately established, so the subsection must end with a direct comparison rather than a list.*

2.4. Variable Neighborhood Search and Hybrid Variants

[TO BE WRITTEN] *Must acknowledge that VNS/VND combinations with probabilistic acceptance are established, and reposition the contribution accordingly: the claim is not a new hybrid template but the neighborhood structures and their empirical validation. Addresses Reviewer 2 comment 1 and Reviewer 4 comment 1.*

3. Problem Definition

The logistics network proposed by Rajabi-Kafshgar et al. (2023), illustrated in Figure 1, describes the flow of products and by-products along the pistachio supply chain. The process begins with the farmers (i), who send the harvested pistachios to the processing centers (j). At these centers, the pistachios are transformed into three main components: open-shell pistachios, raw kernels, and organic waste.

Each of these components follows specific flows. Open-shell pistachios are forwarded to pistachio factories (k), which process and distribute them to final customers (n_1). Raw kernels are sent to oil extraction centers (e), responsible for producing pistachio oil, which is directed to both final customers (n_2) and cosmetic factories (s). These, in turn, process the oil into cosmetic products that are supplied to their respective consumers (n_3). Organic waste generated in the initial processing is sent to composting centers (q), where it is transformed into compost, subsequently distributed to composting customers (m).

Although in practical terms, the produced compost could be reused by the original farmers, establishing a truly closed-loop system, the adopted mathematical model does not explicitly represent this feedback. Compost customers (m) are modeled as distinct entities from the initial farmers (i), thus the feedback cycle is not enforced through model constraints. This simplification, already adopted in previous works (Pishvaee et al., 2010; Ahmadi and Amin, 2019; Banasik et al., 2016), preserves the structural simplicity

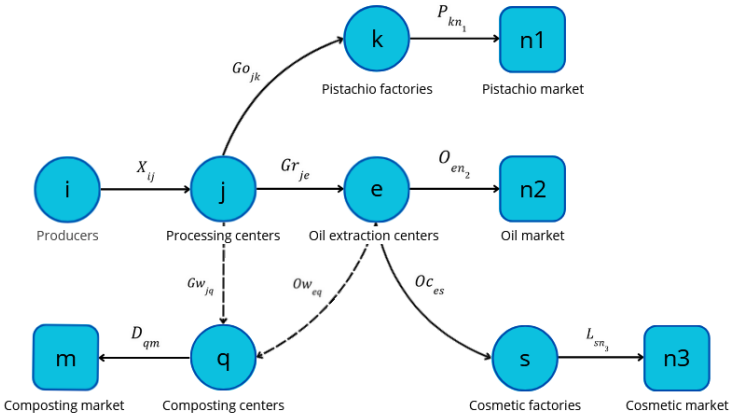


Figure 1: Structure of the proposed pistachio supply chain.

of the model and computational tractability. However, explicitly modeling this return loop with mass balance constraints represents a promising direction for future investigations, especially in the context of sustainable supply chains.

The mixed linear problem aims to minimize total costs (1), including opening, production, and transportation costs, subject to the following capacity and demand constraints, tolerating no shortages. Key model parameters include: β (weight loss percentage during drying operations at processing centers); θ_1, θ_2, and θ_3 (waste percentages of open-shell pistachios, raw kernels, and organic waste, respectively, where $\theta_1 + \theta_2 + \theta_3 = 1$); γ_k, γ_s, and γ_q (production rates for pistachio factories, cosmetic factories, and composting centers, respectively); and λ (waste percentage during oil extraction). A complete description of all parameters and decision variables is provided in Table 5 of Rajabi-Kafshgar et al. (2023).

$$\min\ z_1 + z_2 + z_3 \tag{1}$$

$$\text{s.t.:} \quad \sum_{j=1}^{J} X_{ij} \leq Cpa_i, \quad \forall i \in I \tag{2}$$

$$\sum_{i=1}^{I} X_{ij} \leq Cpu_j \times U_j, \quad \forall j \in J \tag{3}$$

$$\sum_{e=1}^{E} Ow_{eq} + \sum_{j=1}^{J} Gw_{jq} \leq Cpy_q \times Y_q, \quad \forall q \in Q \tag{4}$$

$$\sum_{j=1}^{J} Go_{jk} \leq Cpw_k \times W_k, \quad \forall k \in K \tag{5}$$

$$\sum_{j=1}^{J} Gr_{je} \leq Cpr_e \times R_e, \quad \forall e \in E \tag{6}$$

$$\sum_{e=1}^{E} O_{en_2} \leq Cpv_s \times V_s, \quad \forall s \in S \tag{7}$$

$$\sum_{k=1}^{K} Go_{jk} + \sum_{e=1}^{E} Gr_{je} + \sum_{q=1}^{Q} Gw_{jq} \leq \sum_{i=1}^{I} X_{ij}, \quad \forall j \in J \tag{8}$$

$$\sum_{k=1}^{K} Go_{jk} = (1-\beta) \sum_{i=1}^{I} X_{ij}\theta_1, \quad \forall j \in J \tag{9}$$

$$\sum_{e=1}^{E} Gr_{je} = (1-\beta) \sum_{i=1}^{I} X_{ij}\theta_2, \quad \forall j \in J \tag{10}$$

$$\sum_{q=1}^{Q} Gw_{jq} = \sum_{i=1}^{I} X_{ij}\theta_3, \quad \forall j \in J \tag{11}$$

$$\theta_1 + \theta_2 + \theta_3 = 1 \tag{12}$$

$$\sum_{n_1=1}^{N_1} P_{kn_1} \leq \sum_{j=1}^{J} Go_{jk} \times \gamma_k, \quad \forall k \in K \tag{13}$$

$$\sum_{n_2=1}^{N_2} O_{en_2} + \sum_{s=1}^{S} Oc_{es} \leq \sum_{j=1}^{J} Gr_{je} \times (1-\lambda), \quad \forall e \in E \tag{14}$$

$$\sum_{q=1}^{Q} Ow_{eq} \leq \sum_{j=1}^{J} Gr_{je} \times \lambda, \quad \forall e \in E \tag{15}$$

$$\sum_{n_3=1}^{N_3} L_{sn_1} \leq \sum_{e=1}^{E} Oc_{es} \times \gamma_s, \quad \forall s \in S \tag{16}$$

$$\sum_{m=1}^{M} D_{qm} \leq \left(\sum_{j=1}^{J} Gw_{jq} + \sum_{e=1}^{E} Ow_{eq} \right) \times \gamma_q, \quad \forall q \in Q \tag{17}$$

$$\sum_{k=1}^{K} P_{kn_1} \geq Dp_{n_1}, \quad \forall n_1 \in N_1 \tag{18}$$

$$\sum_{e=1}^{E} O_{en_2} \geq Du_{n_2}, \quad \forall n_2 \in N_2 \tag{19}$$

$$\sum_{s=1}^{S} L_{sn_3} \geq Ds_{n_3}, \quad \forall n_3 \in N_3 \tag{20}$$

$$\sum_{q=1}^{Q} D_{qm} \leq Dc_m, \quad \forall m \in M \tag{21}$$

- The objective function (1) minimizes the total costs of the supply chain, including opening, transportation, and production costs. The costs are divided into three components: z_1 (22), z_2 (23), and z_3 (24), which represent facility opening costs, transportation costs, and production costs, respectively, as shown in the following equations:

$$\begin{aligned} z_1 = & \sum_{j=1}^{J} Fu_j \times U_j + \sum_{q=1}^{Q} Fy_q \times Y_q + \sum_{k=1}^{K} Fw_k \times W_k \\ & + \sum_{e=1}^{E} Fr_e \times R_e + \sum_{s=1}^{S} Fv_s \times V_s \end{aligned} \tag{22}$$

$$
\begin{aligned}
z_2 = & \sum_{i=1}^{I}\sum_{j=1}^{J} CI_i \times X_{ij} + \sum_{j=1}^{J}\sum_{k=1}^{K} Cu_{1j} \times Go_{jk} \\
& + \sum_{j=1}^{J}\sum_{e=1}^{E} Cu_{2j} \times Gr_{je} + \sum_{q=1}^{Q}\sum_{m=1}^{M} Cy_q \times D_{qm} \\
& + \sum_{k=1}^{K}\sum_{n_1=1}^{N_1} Cw_k \times P_{kn_1} \\
& + \sum_{e=1}^{E} Cr_e \times \left(\sum_{n_2=1}^{N_2} O_{en_2} + \sum_{s=1}^{S} Oc_{es} \right) \\
& + \sum_{s=1}^{S}\sum_{n_3=1}^{N_3} Cv_s \times L_{sn_3}
\end{aligned} \tag{23}
$$

$$
\begin{aligned}
z_3 = & \sum_{i=1}^{I}\sum_{j=1}^{J} CX_{ij} \times X_{ij} + \sum_{j=1}^{J}\sum_{k=1}^{K} CK_{jk} \times Go_{jk} \\
& + \sum_{j=1}^{J}\sum_{q=1}^{Q} CJ_{jq} \times Gw_{jq} + \sum_{e=1}^{E}\sum_{s=1}^{S} CS_{es} \times Oc_{es} \\
& + \sum_{e=1}^{E}\sum_{n_2=1}^{N_2} CN_{en_2} \times Oc_{en_2} + \sum_{e=1}^{E}\sum_{q=1}^{Q} CQ_{eq} \times Ow_{eq} \\
& + \sum_{s=1}^{S}\sum_{n_3=1}^{N_3} Cl_{sn_3} \times L_{sn_3} + \sum_{k=1}^{K}\sum_{n_1=1}^{N_1} Cp_{kn_1} \times P_{kn_1} \\
& + \sum_{q=1}^{Q}\sum_{m=1}^{M} Cd_{qm} \times D_{qm}
\end{aligned} \tag{24}
$$

- The variables U_j, Y_q, W_k, R_e, and V_s are binary and determine whether certain elements of the model are active. For example, $U_j = 1$ indicates that processing center j is operational, and $U_j = 0$ otherwise. Similarly, W_k, R_e, V_s, and Y_q indicate whether pistachio factories, oil extraction centers, cosmetic factories, and composting centers are active, respectively. The other variables in the model are continuous and

non-negative, representing real values greater than or equal to zero. These variables model flows, quantities, and other measurable factors within the system.

- Constraints (2)-(7) address the capacity and demand limitations of each facility in the supply chain. In particular, constraint (2) limits the quantity of products transported from farmers to distribution centers. Constraints (3)-(7) ensure that the quantities sent to processing centers, composting centers, pistachio factories, oil extraction centers, and cosmetic factories do not exceed the capacities of these facilities.

- The quantity of products shipped from any facility must not exceed the orders received by that facility. Consequently, constraint (8) establishes that the quantity of products shipped from a processing center must be less than or equal to the quantity received by other facilities. Constraints (9)-(12) define the quantities of open-shell pistachios, raw kernels, and waste generated during the hulling process at each processing center. Similarly, constraints (13)-(17) govern the flows within pistachio factories, oil extraction centers, cosmetic factories, and composting centers.

- Finally, to ensure that customer demand is met, constraints (18)-(21) ensure that the quantities of products delivered to pistachio, pistachio oil, cosmetics, and compost customers satisfy or exceed their respective demands.

Having defined the mathematical formulation of the pistachio supply chain optimization problem, we now present our solution methodology based on Variable Neighborhood Search with problem-specific operators designed to exploit the structure of this complex multi-echelon network.

4. Methodology

This section details the proposed Variable Neighborhood Search (VNS) algorithm with priority-based encoding. Section 4.1 presents the integer-based chromosome representation and Section 4.2 the backward decoding strategy that maps it onto a feasible transportation tree. Section 4.3 then defines the problem-specific neighborhood structures designed to exploit the pistachio supply chain's structural properties, and Section 4.4 describes how the search procedure combines them.

4.1. Solution Representation

Representing a solution for a complex network design problem is non-trivial. Following the principles established by (Gen et al., 2006) and Rajabi-Kafshgar et al. (2023), we employ a priority-based encoding method. In this scheme, the chromosome does not directly encode the flow quantities (X_{ij}), but rather the *priority* of serving specific transportation arcs.

4.1.1. Integer Permutation vs. Real-Valued Encoding

In this work, we use an integer permutation encoding rather than the random key ($0 - 1$ real value) encoding used in previous studies (Rajabi-Kafshgar et al., 2023; Fathollahi-Fard and Hajiaghaei-Keshteli, 2018).

In the real-valued approach, genes are assigned random numbers $r \in [0, 1]$, and priorities are derived from sorting these values. This representation suffers from a "many-to-one" mapping redundancy: infinite combinations of real numbers can result in the exact same priority order. For example, the vectors $v_1 = [0.1, 0.2, 0.5]$ and $v_2 = [0.8, 0.85, 0.9]$ both map to the same permutation indices $(1, 2, 3)$. This redundancy creates "flat" regions in the search landscape where small perturbations (mutations) in the genotype produce no change in the phenotype (the solution structure).

In contrast, we employ sequential integers from 1 to Z, where Z is the total number of candidate source-depot pairs in a specific chain segment. This ordinal representation ensures a one-to-one mapping between the genotype and the solution structure. This property is crucial for the VNS algorithm, as it ensures that every neighborhood move results in a distinct solution configuration, effectively eliminating redundant search steps.

4.1.2. Chromosome Structure

The solution is represented by a composite chromosome divided into eight distinct segments (S_1 to S_8), as illustrated in Fig. 2. Each segment contains a permutation of integers representing the transport priorities for a specific part of the supply chain.

4.2. Decoding Procedure

The decoding process serves as a constructive heuristic that transforms the priority chromosome into a feasible transportation tree. As shown in Fig. 3, the decoding process begins with the flow of final products (pistachios, oil, cosmetics) from factories to consumers, working backward through

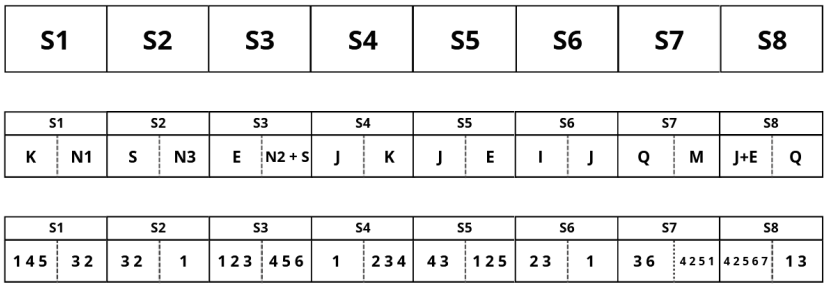


Figure 2: Structure of the priority-based chromosome. The solution is composed of 8 distinct integer permutations ($S_1 - S_8$), each governing the transport priority of a specific subsection in the supply chain.

the supply chain. This backward approach is preferred over forward decoding because it naturally respects demand constraints: consumer demand is satisfied first, propagating the requirements upstream to factories and processing centers. This strategy guarantees feasibility by ensuring that facility capacities are respected at every stage while preventing overproduction, a critical consideration in perishable agricultural supply chains.

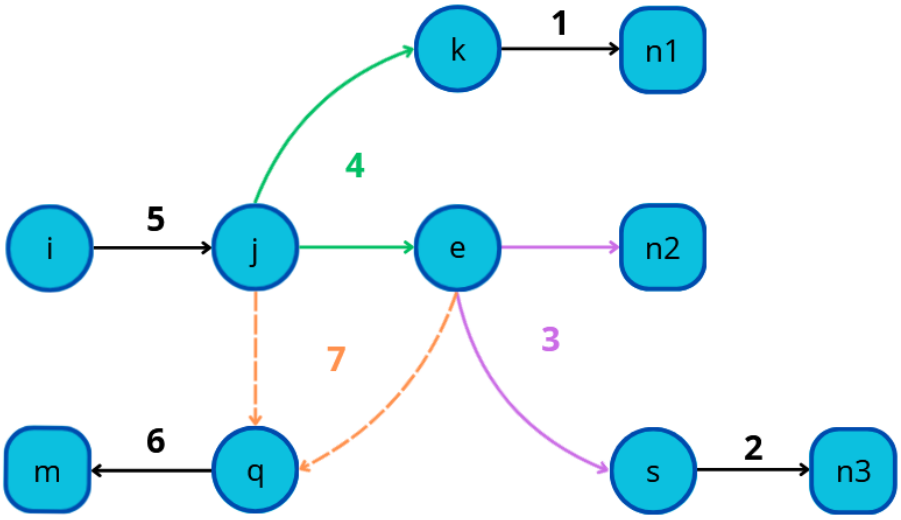


Figure 3: The backward decoding flow. The process prioritizes satisfying end-consumer demands first to propagate requirements upstream, ensuring feasibility throughout the chain.

As illustrated in Figure 3, the decoding process initiates with the flow of derived pistachio products from factories to consumers, ensuring all constraints are satisfied. Oil and cosmetic product flows follow similar principles. Processing centers regulate pistachio allocation, accounting for production losses and ensuring viable distribution. Harvested pistachios are allocated proportionally between factories and oil extraction centers, prioritizing demand satisfaction and minimizing transportation costs. Flows between producers and processing centers cannot exceed production capacity, guaranteeing system feasibility. Waste flows are managed similarly, respecting processing and composting capacity limits while minimizing costs.

To decode each logistic flow in the supply chain, we employ the `decodingStep` function, which implements the priority-based transportation algorithm described by Gen et al. (2006). This function acts iteratively, allocating product units from source to destination based on a chromosome vector that defines the priority order among logistic nodes, always respecting source capacities, destination demands, and unit transportation costs. The `decodingStep` function corresponds directly to the priority-based procedure detailed in Table 2 of Gen et al. (2006).

Algorithms 1 and 2 detail the complete decoding process in two phases (separated due to page constraints). Algorithm 1 initiates the process by decoding the main flows between factories, extraction centers, consumers, and processing centers using chromosome segments S_1 through S_5. The critical logic occurs in Steps 5 and 6, where the algorithm reconciles diverging flows from Processing Centers, which produce multiple outputs (Kernels for Oil vs. Open-shell for Factories) in fixed proportions. Satisfying one downstream demand often creates surplus of the other output. The decoding identifies the bottleneck demand and routes surplus to the lowest-cost destination, minimizing over-supply penalties. Critically, this reconciliation mechanism explicitly enforces the joint production equality constraints (9) and (10). After initial flows are allocated, the algorithm computes the implied input requirement from each Processing Center under both scenarios (if G_o dictates capacity vs. if G_r dictates capacity), selects the binding constraint, and augments the non-binding output flow to maintain the exact proportions $(1-\beta)\theta_1$ and $(1-\beta)\theta_2$. This adjustment step transforms inequality-based greedy allocation into an equality-satisfying solution, ensuring that the mass balance at Processing Centers rigorously conforms to the fixed conversion ratios defined by the waste percentages.

Algorithm 2 performs additional adjustments to ensure consistency be-

tween inputs and outputs at intermediate centers, decodes transport from producers to processing centers (segment S_6), allocates waste generated in the processes to composting centers (segments S_7 and S_8), and determines which facilities should be opened based on the obtained flows. At the end of the process, a complete set of continuous and binary decision variables is obtained, representing a feasible and coherent solution to the proposed optimization problem.

4.3. Neighborhood Structures

A critical inefficiency in applying standard permutation operators to priority-based encoding is their "blindness" to the phenotypic state. In the greedy decoding process (Algorithm 1), typically only a subset of high-priority nodes is required to satisfy all demands. We term these Active Nodes ($\mathcal{A}$), while the remaining unused nodes are Inactive ($\mathcal{I}$). Standard operators often swap two Inactive nodes, changing the genotype but resulting in the exact same transportation tree and objective value.

To overcome this redundancy, our implementation explicitly tracks the set of active nodes returned by the decoder and employs four structure-aware operators. These operators are designed to either intensify the search within the current topology or diversify it by forcing interactions between the active and inactive sets:

- **Active Reversion:** This operator functions as an intensification mechanism. It selects a segment of the chromosome containing multiple nodes from set $\mathcal{A}$ and reverses their priority values. By reordering only the suppliers that are already in use, this move fine-tunes the flow distribution to find lower-cost allocations without necessarily opening new routes or altering the network structure.

- **Inactive-Active Swap:** This is a global diversification operator. It selects one active node ($i \in \mathcal{A}$) and one inactive node ($j \in \mathcal{I}$) at random and exchanges their priorities. This forces a currently used route to be downgraded and a previously unused route to be upgraded, effectively testing global topology changes by bringing new nodes into the solution.

- **Inactive Boost:** This operator addresses the "starvation" issue common in capacity-constrained problems. A low-cost supplier might be

Algorithm 1 Chromosome decoding for the supply chain (Part 1)

procedure DECODE(data)
    Input: structured data containing:
    - Capacities ($\mathbf{C}$) and demands ($\mathbf{D}$) for all facilities
    - Unit transportation costs ($\mathbf{T}$)
    - Loss rates for each process stage
    - Consumer demands
    - Chromosome segments ($S_1, S_2, \ldots, S_8$) representing transport priorities
    Output: TotalCost, Decision Variables ($\mathbf{X}, \mathbf{P}, \mathbf{L}, \mathbf{O}, \ldots$)
    // Step 1: Initialize variables
    $TotalCost \leftarrow 0$
    // Step 2: Decode Pistachio Factories → Pistachio Consumers
    $a_1 \leftarrow$ capacities of pistachio factories
    $b_1 \leftarrow$ demands of pistachio consumers
    $c_1 \leftarrow$ transportation costs
    $(cost1, \mathbf{P}) \leftarrow \text{decodingStep}(S_1, a_1, b_1, c_1)$
    $TotalCost \leftarrow TotalCost + cost1$
    // Step 3: Decode Cosmetic Factories → Cosmetic Consumers
    $a_2 \leftarrow$ capacities of cosmetic factories
    $b_2 \leftarrow$ demands of cosmetic consumers
    $c_2 \leftarrow$ transportation costs
    $(cost2, \mathbf{L}) \leftarrow \text{decodingStep}(S_2, a_2, b_2, c_2)$
    $TotalCost \leftarrow TotalCost + cost2$
    // Step 4: Decode Oil Extraction Centers → Consumers
    $a_3 \leftarrow$ capacities of oil extraction centers
    $b_3 \leftarrow$ demands of oil consumers + cosmetic factories
    $c_3 \leftarrow$ transportation costs
    $(cost3, OO_c) \leftarrow \text{decodingStep}(S_3, a_3, b_3, c_3)$
    Separate OO_c into $\mathbf{O}$ (oil consumers) and O_c (cosmetic factories)
    $TotalCost \leftarrow TotalCost + cost3$
    // Step 5: Decode Processing Center → Pistachio Factories
    $a_4 \leftarrow$ capacities of processing centers
    $b_4 \leftarrow$ adjusted demands of pistachio factories
    $c_4 \leftarrow$ transportation costs
    $(\_\_, G_o) \leftarrow \text{decodingStep}(S_4, a_4, b_4, c_4)$
    // Step 6: Decode Processing Center → Oil Extraction Centers
    $a_5 \leftarrow$ capacities of processing centers
    $b_5 \leftarrow$ adjusted demands of oil extraction centers
    $c_5 \leftarrow$ transportation costs
    $(\_\_, G_r) \leftarrow \text{decodingStep}(S_5, a_5, b_5, c_5)$
end procedure

Algorithm 2 Chromosome decoding (Part 2): Upstream flows and waste management

procedure DECODE(continuation from Algorithm 1)
    // Step 7: Adjust demands at Processing Centers
    for each processing center j **do**
        Calculate required outputs based on pistachio and oil flows
        Update G_o and G_r to match effective capacities
    end for
    // Step 8: Decode Pistachio Farmers → Processing Centers
    $a_6 \leftarrow$ capacities of pistachio farmers
    $b_6 \leftarrow$ updated demands at processing centers
    $c_6 \leftarrow$ transportation costs
    $(cost6, \mathbf{X}) \leftarrow \text{decodingStep}(S_6, a_6, b_6, c_6)$
    $TotalCost \leftarrow TotalCost + cost6$
    // Step 9: Decode Composting Centers → Compost Consumers
    $a_7 \leftarrow$ capacities of composting centers
    $b_7 \leftarrow$ demands of compost consumers
    $c_7 \leftarrow$ transportation costs
    $(cost7, \mathbf{D}) \leftarrow \text{decodingStep}(S_7, a_7, b_7, c_7)$
    $TotalCost \leftarrow TotalCost + cost7$
    // Step 10: Assign waste flows
    Calculate waste from processing centers (J) and Oil Centers (E)
    $a_8 \leftarrow$ available capacities of composting centers
    $b_8 \leftarrow$ waste demands to be disposed
    $c_8 \leftarrow$ transportation costs
    $(\_, O_w) \leftarrow \text{decodingStep}(S_8, a_8, b_8, c_8)$
    Update $TotalCost$
    // Step 11: Define binary opening variables
    Set $\mathbf{U}, \mathbf{W}, \mathbf{R}, \mathbf{V}, \mathbf{Y} = 1$ if corresponding flow > 0
    Add fixed costs to $TotalCost$
    return $TotalCost$, Decision Variables, Binary Variables
end procedure

ignored simply because it sits too low in the priority list to be considered before demands are met. This operator selects a random inactive node and assigns it the highest possible priority value in the chromosome. This forces the greedy decoder to maximize flow from this specific node before considering any others, potentially revealing a completely different and superior tree structure.

- **Inactive-Active Exchange two Neighbors (IA-ETN):** This operator provides a "localized" diversification. Instead of a random global swap, it selects an inactive node and swaps it with its nearest active neighbor in terms of chromosome index distance. Unlike the global swap, which can be highly disruptive, this operator explores the "boundary" of the solution, testing if a route that is "almost" good enough (close to the active set) should replace a marginally active route.

4.4. The VNS Algorithm

The proposed algorithm is a hybrid metaheuristic that enhances the standard Variable Neighborhood Search (VNS) (Hansen et al., 2010) by integrating Variable Neighborhood Descent (VND) for intensification and a Simulated Annealing (SA) acceptance criterion. This hybridization addresses the limitations of pure descent methods in the highly constrained landscape of the pistachio supply chain. The procedure is outlined in Algorithm 3.

The intensification phase employs a Variable Neighborhood Descent (VND) strategy rather than a simple local search. In this step, the algorithm iterates sequentially through the entire set of neighborhood structures. Whenever an improvement is found with any operator, the search restarts from the first operator, ensuring that the resulting solution is a local optimum with respect to all defined neighborhoods.

To prevent stagnation in local optima, we replace the standard deterministic acceptance criterion with a probabilistic Simulated Annealing mechanism. This probabilistic acceptance is particularly valuable in highly constrained problems like supply chain networks, where the solution space contains numerous local optima separated by infeasible regions. A new solution generated by the VND phase is accepted as the current solution if it improves the objective function or, if it is worse, with a probability given by the Boltzmann factor $P = e^{-\Delta/T}$, where Δ is the deterioration in solution quality and T is the current temperature. This allows the search to traverse

valleys in the fitness landscape and explore promising regions that would be rejected by pure descent methods.

Finally, the shaking phase applies a perturbation to the current solution to escape the basin of attraction of the local optimum. The magnitude of this perturbation is adaptive, defined in our implementation as 7% of the total decision variables in the chromosome. This value was determined through preliminary experiments testing perturbation strengths ranging from 3% to 15% on representative instances. Strengths below 5% resulted in insufficient diversification, causing the algorithm to revisit previously explored regions, while values above 10% caused excessive disruption, degrading solution quality. The 7% value provided the best balance between escaping local optima and maintaining solution structure. The specific operator used for shaking changes cyclically, resetting to the first operator upon a successful acceptance and moving to the next operator if the candidate solution is rejected.

The VND phase applies the four operators defined in Section 4.3 in the following fixed sequence: (1) Active Reversion, (2) Inactive-Active Swap, (3) Inactive Boost, and (4) Inactive-Active Exchange two Neighbors (IA-ETN). This specific ordering follows a strategic progression from conservative to aggressive exploration. First, Active Reversion fine-tunes the existing solution topology by reordering active nodes, exploring the current structure before considering more disruptive changes. Second, Inactive-Active Swap introduces global diversification by randomly exchanging nodes between active and inactive sets, testing whether alternative facility configurations might be superior. Third, Inactive Boost applies the most aggressive perturbation, forcing a previously unused node to maximum priority—this operator is crucial for escaping situations where good facilities are trapped at low priorities. Finally, IA-ETN provides localized diversification by testing the boundary between active and inactive sets, swapping neighboring nodes to explore incremental topology changes. This gradual escalation from refinement to disruption ensures that the algorithm exhausts local improvements before resorting to more radical modifications, maximizing the effectiveness of the VND phase.

These operators are applied to the priority-based chromosome during the VND phase, with each move triggering a complete re-decoding using the procedure described in Algorithms 1 and 2.

Algorithm 3 Hybrid VNS with SA Acceptance

Require: Initial solution x, Operators set $\mathcal{N}$, Max evaluations N_{max}, Initial Temp T_0, Cooling rate α

Output: Best solution found x^*
$x_{curr} \leftarrow x$; $\quad x^* \leftarrow x$
$n_{eval} \leftarrow 1$; $\quad T \leftarrow T_0$
$k \leftarrow 0$ ▷ Operator index for shaking
while $n_{eval} < N_{max}$ **do**
    // Phase 1: Intensification (VND)
    $x_{new} \leftarrow \text{VND\_LocalSearch}(x_{curr}, \mathcal{N})$
    Update n_{eval}
    // Phase 2: Update Global Best
    if $f(x_{new}) < f(x^*)$ **then**
        $x^* \leftarrow x_{new}$
    end if
    // Phase 3: Acceptance Criteria (SA)
    $\Delta \leftarrow f(x_{new}) - f(x_{curr})$
    if $\Delta \leq 0$ **or** $\text{Random}(0,1) < e^{-\Delta/T}$ **then**
        $x_{curr} \leftarrow x_{new}$
        $k \leftarrow 0$ ▷ Reset shaking operator on acceptance
    else
        $k \leftarrow (k+1) \pmod{|\mathcal{N}|}$ ▷ Next shaking operator
    end if
    // Phase 4: Perturbation and Cooling
    $T \leftarrow T \times \alpha$
    $x_{curr} \leftarrow \text{Shake}(x_{curr}, \mathcal{N}[k])$
end while

5. Computational Experiments

This section evaluates the performance of the proposed VNS approach. Sections 5.1 to 5.4 describe the computational environment, the benchmark instances, the components shared by all compared algorithms, and the statistical methodology. Section 5.5 then benchmarks the proposed VNS against a Genetic Algorithm (GA), an Iterated Local Search (ILS), and the exact solver. Section 5.6 isolates the contribution of the problem-specific neighborhood structures by comparing them against classical permutation operators, and Section 5.7 reports an ablation study that separates the contribution of each individual component of the proposed algorithm.

5.1. Experimental Setup

The algorithms were implemented in Python and executed on a workstation with an Intel Xeon E3-1240 V2 processor (8 cores) and 32 GB of RAM running Ubuntu 24.04 LTS. The exact method was solved using Gurobi Optimizer v11.0.

5.2. Benchmark Instances

The test instances were generated following the methodology proposed by Rajabi-Kafshgar et al. (2023). Each instance is characterized by equal-sized entities across all echelons (i.e., $I = J = K = E = S = Q = N_1 = N_2 = N_3 = M$), with instance sizes of 10, 30, 100, 200, 400, 800, and 1600. Since the chromosome consists of eight segments encoding all possible source-destination pairs ($I \times J$, $J \times K$, $J \times E$, $K \times N_1$, $E \times N_2$, $E \times S$, $S \times N_3$, and $Q \times M$), this results in chromosome lengths ranging from 1,700 (for $N = 10$) to 27,200 (for $N = 1600$) variables in the priority-based encoding.

All problem parameters (facility costs, production costs, transportation costs, capacities, demands, and conversion rates) were randomly generated using uniform distributions with ranges identical to those specified in Table 7 of Rajabi-Kafshgar et al. (2023). Complete parameter specifications and the generated instances are publicly available in the project repository to ensure full reproducibility.

5.3. Algorithm Configurations

5.3.1. Shared Components

[TO BE WRITTEN] *Table of shared versus algorithm-specific components, establishing that the three metaheuristics differ only in their search strategy.*

To ensure a fair comparison across all metaheuristics, each algorithm was allocated an identical computational budget: $N_{max} = 10n$, where n represents the total number of decision variables in the priority-based encoding. This budget scales linearly with problem size, ensuring equitable evaluation across all instances.

All metaheuristics (VNS, GA, ILS) were executed 30 times for each instance to ensure statistical robustness.

5.3.2. Algorithm-Specific Parameters

The specific parameter settings for each algorithm are as follows:

- **Genetic Algorithm (GA):** The GA uses a population size of $P = \lfloor\sqrt{n}\rfloor$, where n is the total number of decision variables in the priority encoding, motivated by population-sizing theory suggesting that required population grows roughly with the square root of problem size. The algorithm employs a hybrid crossover operator (probability 0.9) that randomly alternates between two strategies with equal probability: (i) segment-level crossover, which exchanges entire chromosome segments (S_1 through S_8) between parents, and (ii) intra-segment crossover, which applies Partially Mapped Crossover (PMX) within each individual segment to preserve permutation validity while promoting diversity. This hybrid approach balances global exploration (segment exchange) with local refinement (intra-segment recombination). Mutation probability is set to 0.1, and binary tournament selection is used for parent selection, following widely established GA practices (Hussain et al., 2022).

- **Iterated Local Search (ILS):** The ILS alternates between local search and perturbation phases. The perturbation strength scales with instance size, and the algorithm restarts when no improvement is observed after 10 consecutive non-improving iterations (Tinós et al., 2024). Local search employs the problem-specific "Inactive–Active Swap" operator.

- **Variable Neighborhood Search (VNS):** The proposed VNS uses initial temperature $T_0 = 100$ and cooling rate $\alpha = 0.995$, which are standard parameters widely adopted in Simulated Annealing literature (Kirkpatrick et al., 1983), providing effective balance between exploration and exploitation. The perturbation strength is set to 7% of

the chromosome length, a value empirically determined as described in Section 4.4. The VND phase cycles sequentially through four problem-specific operators: (1) Active Reversion, (2) Inactive-Active Swap, (3) Inactive Boost, and (4) IA-ETN. Upon finding an improvement with any operator, the VND restarts from the first operator.

- **Exact Solver (Gurobi):** For the three largest instances (400, 800, and 1600), the solver was configured with a time limit of 3,600 seconds (1 hour). For smaller instances, no time limit was imposed, allowing the solver to run until proven optimality. For instances where the optimal solution could not be found within the time limit, the best feasible solution found (Incumbent) was recorded.

5.4. Statistical Methodology

The same statistical procedure is applied to every pairwise comparison reported in this section. Since parametric tests (such as the t-test) assume normally distributed data, we first applied the Shapiro-Wilk test with significance level $\alpha = 0.05$ to check this assumption. When both samples passed the normality test ($p > 0.05$), we used the parametric t-test, which offers higher statistical power. When either sample failed the normality test ($p \leq 0.05$), we used the non-parametric Mann-Whitney U test, which makes no distributional assumptions. All hypotheses are one-tailed, testing whether the first method attains a smaller (better) objective function value than the second.

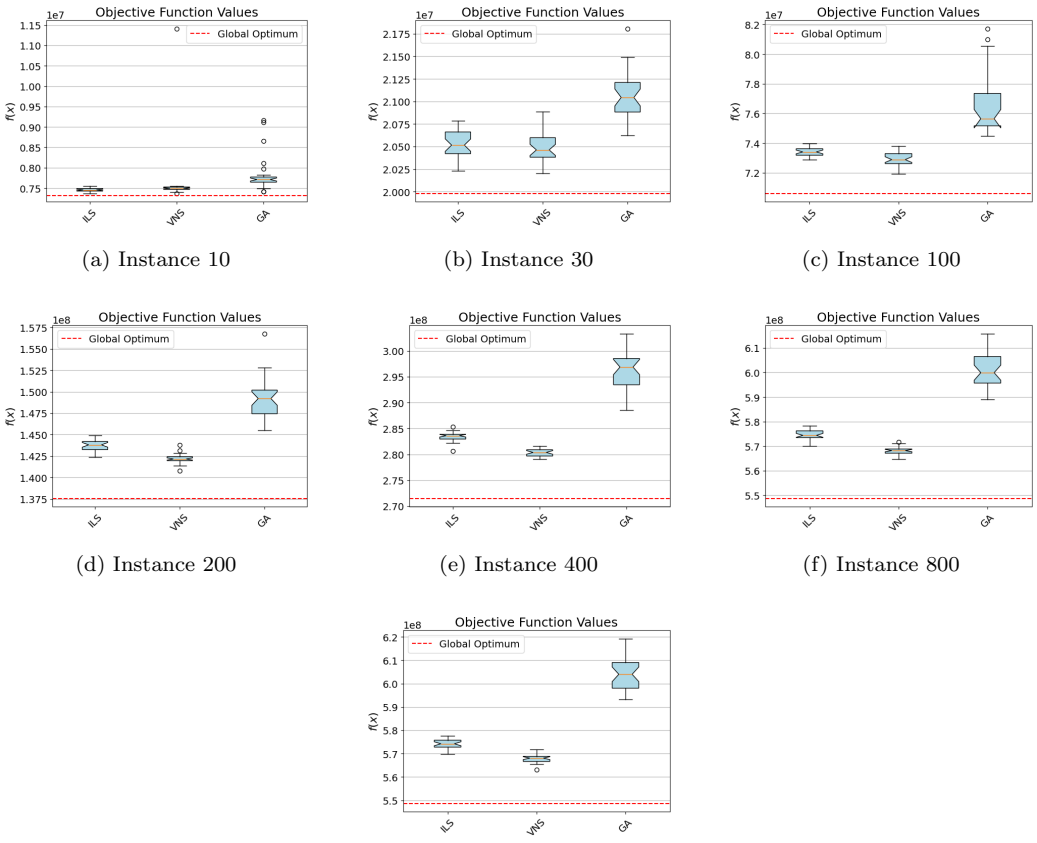
5.5. Comparative Analysis

In this section, we provide a comprehensive benchmark of the proposed VNS against the Genetic Algorithm (GA), Iterated Local Search (ILS), and the exact solver (Gurobi). Fig. 4 presents the boxplots for all methods across the tested instances. The red dashed line represents the optimal solution (or best lower bound) found by Gurobi.

5.5.1. Performance Comparison

Table 1 summarizes the average objective function values and the relative gap to the optimal solution for each method.

For the smallest instance (10), the ILS achieves the best average performance, surpassing the VNS by 2.21%. This is expected, as simple local search strategies often converge faster in very small search spaces where deep



(a) Instance 10

(b) Instance 30

(c) Instance 100

(d) Instance 200

(e) Instance 400

(f) Instance 800

(g) Instance 1600

Figure 4: Comparison of VNS, ILS, and GA across all instance sizes. The red dashed line indicates the reference solution (Gurobi optimum or best incumbent).

exploration is unnecessary. For Instance 30, the performance of VNS and ILS is statistically indistinguishable.

However, a clear trend emerges from Instance 100 onwards. The VNS consistently outperforms both ILS and GA, with the performance gap widening as the problem complexity increases. For the largest instances (800 and 1600), the VNS maintains a gap to the optimum of approximately 3.5%, whereas the ILS gap grows to 4.7% and the GA gap exceeds 10%. This demonstrates the superior scalability of the proposed approach.

For the three largest instances (400, 800, and 1600), Gurobi was subject to a one-hour time limit to prevent excessive computational costs. Instance 400 reached proven optimality within this limit. However, Instances 800 and 1600 exhausted the time budget without proving optimality, returning the same best incumbent solution value of 5.49×10^8 for both. These values represent the best feasible solutions found (upper bounds) rather than proven optimal solutions, as the optimality gap had not closed. For smaller instances (10, 30, 100, and 200), Gurobi ran without time constraints and successfully proved optimality.

Table 1: Descriptive Statistics: Average Objective Function values and Gap to Reference Solution (%). Note: Instances 10-200 were solved to proven optimality by Gurobi without time constraints. Instance 400 reached proven optimality within the 1-hour time limit. For instances 800 and 1600, Gurobi reached the time limit without proving optimality; gaps are calculated relative to the best incumbent solution rather than a proven optimum.

	Reference	**Average Objective Function**			**Gap to Reference (%)**		
Inst.	Gurobi (Opt/Bound)	Proposed VNS	ILS	GA	VNS	ILS	GA
10	7.32×10^6	7.62×10^6	$\mathbf{7.46 \times 10^6}$	7.82×10^6	4.17%	**1.92%**	6.89%
30	2.00×10^7	$\mathbf{2.05 \times 10^7}$	$\mathbf{2.05 \times 10^7}$	2.11×10^7	**2.65%**	2.76%	5.39%
100	7.06×10^7	$\mathbf{7.29 \times 10^7}$	7.34×10^7	7.67×10^7	**3.28%**	3.97%	8.59%
200	1.38×10^8	$\mathbf{1.42 \times 10^8}$	1.44×10^8	1.49×10^8	**3.40%**	4.48%	8.48%
400	2.71×10^8	$\mathbf{2.80 \times 10^8}$	2.83×10^8	2.96×10^8	**3.27%**	4.42%	9.02%
800	5.49×10^8	$\mathbf{5.68 \times 10^8}$	5.75×10^8	6.01×10^8	**3.54%**	4.72%	9.53%
1600	5.49×10^8	$\mathbf{5.68 \times 10^8}$	5.74×10^8	6.04×10^8	**3.50%**	4.65%	10.13%

5.5.2. Statistical Validation

To rigorously validate the performance differences, we applied the statistical methodology described in Section 5.4 with $\alpha = 0.05$ for all tests. Table 2 presents the normality checks (p_{SW}) and hypothesis testing results for all pairwise comparisons between VNS and its competitors.

The hypothesis tested was $H_1 : \mu_{VNS} < \mu_{Competitor}$ (one-tailed). As shown in the table, the data distribution characteristics vary with instance size, requiring different statistical treatments (parametric t-test when normality holds, non-parametric Mann-Whitney U test otherwise).

Table 2: Detailed Statistical Analysis: Normality Checks and Hypothesis Testing ($\alpha = 0.05$).

	Normality Check (p_{SW})			**Hypothesis Testing** ($H_1 : VNS < Competitor$)				
Inst.	VNS	ILS	GA	Comparison	Normality?	Test Used	p-value	**Conclusion**
10	1.70e-11	5.41e-01	3.74e-07	VNS vs ILS	No	Mann-Whitney	9.98e-01	No Sig. Diff.
				VNS vs GA	No	Mann-Whitney	1.09e-07	VNS Superior
30	6.26e-02	6.64e-01	4.62e-01	VNS vs ILS	Yes	t-test	2.94e-01	No Sig. Diff.
				VNS vs GA	Yes	t-test	1.83e-12	VNS Superior
100	5.12e-01	6.16e-01	3.46e-05	VNS vs ILS	Yes	t-test	7.94e-06	VNS Superior
				VNS vs GA	No	Mann-Whitney	1.51e-11	VNS Superior
200	4.34e-02	8.77e-01	6.67e-02	VNS vs ILS	No	Mann-Whitney	3.06e-10	VNS Superior
				VNS vs GA	No	Mann-Whitney	1.51e-11	VNS Superior
400	2.96e-01	2.95e-01	6.25e-01	VNS vs ILS	Yes	t-test	1.15e-20	VNS Superior
				VNS vs GA	Yes	t-test	1.03e-20	VNS Superior
800	7.11e-01	8.82e-01	4.39e-01	VNS vs ILS	Yes	t-test	4.78e-20	VNS Superior
				VNS vs GA	Yes	t-test	1.71e-24	VNS Superior
1600	3.56e-01	1.91e-01	1.27e-01	VNS vs ILS	Yes	t-test	1.38e-17	VNS Superior
				VNS vs GA	Yes	t-test	5.97e-23	VNS Superior

The analysis confirms two distinct performance regimes. In small instances (10-30), the simple ILS is highly efficient, and VNS provides no statistical advantage (p-value > 0.05). However, a transition occurs at Instance 100. From this point onward, the VNS is statistically superior to both ILS and GA ($p < 10^{-5}$). Notably, in Instance 200, the VNS distribution deviated from normality ($p = 4.34 \times 10^{-2}$), requiring the non-parametric Mann-Whitney test, which still confirmed the result with high confidence.

5.5.3. Performance Regime Analysis

The observed performance stratification reveals important insights about algorithm selection based on problem scale. For small instances (10-30 nodes), the solution space is relatively compact, and simple greedy descent strategies employed by ILS converge rapidly to near-optimal solutions. In these cases, the overhead of VNS's multi-neighborhood framework and SA acceptance mechanism provides no advantage, since the problem is simple enough that basic local search suffices. The VND cycling through multiple

operators and the probabilistic acceptance of worse solutions become unnecessary computational expenses when the local optimum basin is shallow and easily escapable.

However, as instance size grows beyond 100 nodes, the combinatorial explosion creates a rugged fitness landscape with numerous deep local optima separated by large barriers. Here, the VNS's sophisticated diversification mechanisms (strategic neighborhood sequencing, SA-based hill climbing, and adaptive perturbation) become essential. The problem-specific operators exploit structural properties of the supply chain that are invisible to classical operators, enabling the algorithm to navigate the active/inactive node boundary more effectively. The ILS, constrained to a single neighborhood operator, lacks this structural awareness and becomes trapped in inferior local optima. Similarly, the GA's population-based search suffers from the curse of dimensionality: as chromosome length grows from 1,700 to 27,200 variables, maintaining population diversity while guiding convergence becomes increasingly difficult, resulting in premature convergence to suboptimal regions.

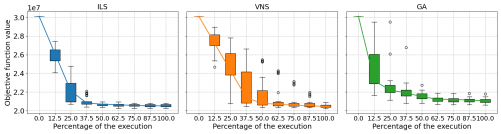
5.5.4. Convergence Analysis

Figure 5 illustrates the evolutionary boxplots for all three methods. The ILS (left column) demonstrates a consistent behavior with relatively low variance (small box sizes) throughout the execution. However, despite this stability, it fails to reach the deeper regions found by the VNS, stopping at higher objective function values.

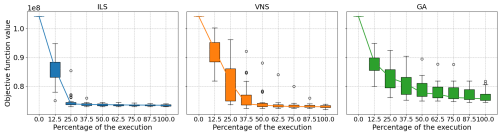
The GA (right column) struggles with the scale of the problem. In larger instances (800 and 1600), its convergence curve is still strictly descending at the end of the computational budget, indicating that it requires significantly more evaluations to compete with the trajectory-based methods.

The Proposed VNS (middle column) offers the best trade-off. In the largest instances, it starts with a broader search distribution (driven by the Simulated Annealing acceptance criteria and the specific neighborhood operators) allowing it to filter through various basins of attraction. Crucially, while the ILS stabilizes early at a sub-optimal level, the VNS sustains a steep descent trajectory for longer, ultimately securing the lowest objective function values with high reliability across all 30 runs.

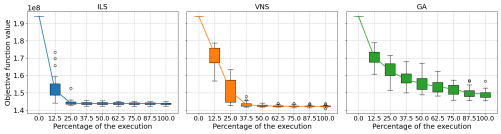
(a) Instance 10



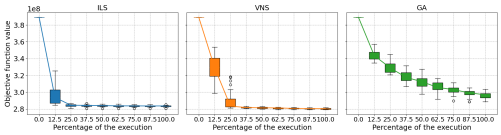
(b) Instance 30



(c) Instance 100

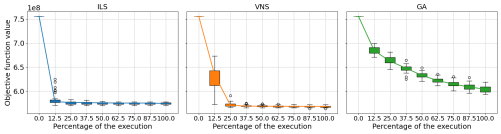


(d) Instance 200



(e) Instance 400

(f) Instance 800



(g) Instance 1600

Figure 5: Convergence profiles of VNS, ILS, and GA across all instance sizes. Boxplots show the distribution of objective values at different execution stages (as percentage of total evaluations), with median lines connecting the boxes. The VNS demonstrates sustained improvement throughout execution, while GA plateaus earlier in large instances.

5.6. Assessment of Neighborhood Structures

To validate the contribution of the novel problem-specific neighborhood structures, we conducted a comparative study against a baseline "Standard VNS". The Standard VNS employs only the classical permutation operators found in the literature: *Swap*, *Reversion*, *Insertion*, and *Slide* (Gendreau and Potvin, 2010).

5.6.1. Statistical Analysis

Following the procedure defined in Section 5.4, the hypothesis tested here was that the Proposed VNS provides a smaller (better) objective function value than the Standard VNS ($H_1 : \mu_{Proposed} < \mu_{Standard}$).

Table 3 details the p-values obtained. For small instances (10 and 30), the p-values (> 0.05) indicate that there is no statistically significant difference between the Proposed VNS and the Standard VNS. The null hypothesis ($H_0 : \mu_{Novel} = \mu_{Basic}$) cannot be rejected, meaning both approaches perform comparably in these simple landscapes. However, for all instances of size 100 and above, the p-values are effectively zero ($p < 0.0001$), confirming that the Proposed VNS is statistically superior with high confidence.

Table 3: Statistical tests comparing Proposed VNS vs. Standard VNS. (S-W = Shapiro-Wilk p-value).

Inst.	S-W (Novel)	S-W (Basic)	Test	p-value	Result
10	1.70×10^{-11}	0.0008	Mann-W.	1.0000	No Sig. Diff.
30	0.0626	0.0493	Mann-W.	0.9811	No Sig. Diff.
100	0.5115	0.5658	t-test	3.64×10^{-6}	Novel Superior
200	0.0434	0.5143	Mann-W.	2.25×10^{-11}	Novel Superior
400	0.2958	0.6396	t-test	2.29×10^{-32}	Novel Superior
800	0.7107	0.1993	t-test	2.76×10^{-27}	Novel Superior
1600	0.3556	0.3454	t-test	5.89×10^{-31}	Novel Superior

5.6.2. Performance Magnitude

Table 4 quantifies the difference in solution quality. While standard operators are sufficient for small problems, their performance degrades as the search space expands. The proposed operators demonstrate a clear scalability advantage, with the performance gap widening in favor of the Proposed VNS as the instance size increases, reaching nearly 5% improvement in the largest instances.

Table 4: Average Objective Function comparison. Gap positive indicates Proposed VNS is better.

Instance	Avg. Obj (Novel)	Avg. Obj (Basic)	Gap (%)
10	7.62×10^6	7.38×10^6	-3.05%
30	2.05×10^7	2.04×10^7	-0.44%
100	7.29×10^7	7.34×10^7	+0.71%
200	1.42×10^8	1.44×10^8	+1.45%
400	2.80×10^8	2.87×10^8	+2.52%
800	5.68×10^8	5.94×10^8	+4.58%
1600	5.67×10^8	5.95×10^8	+4.83%

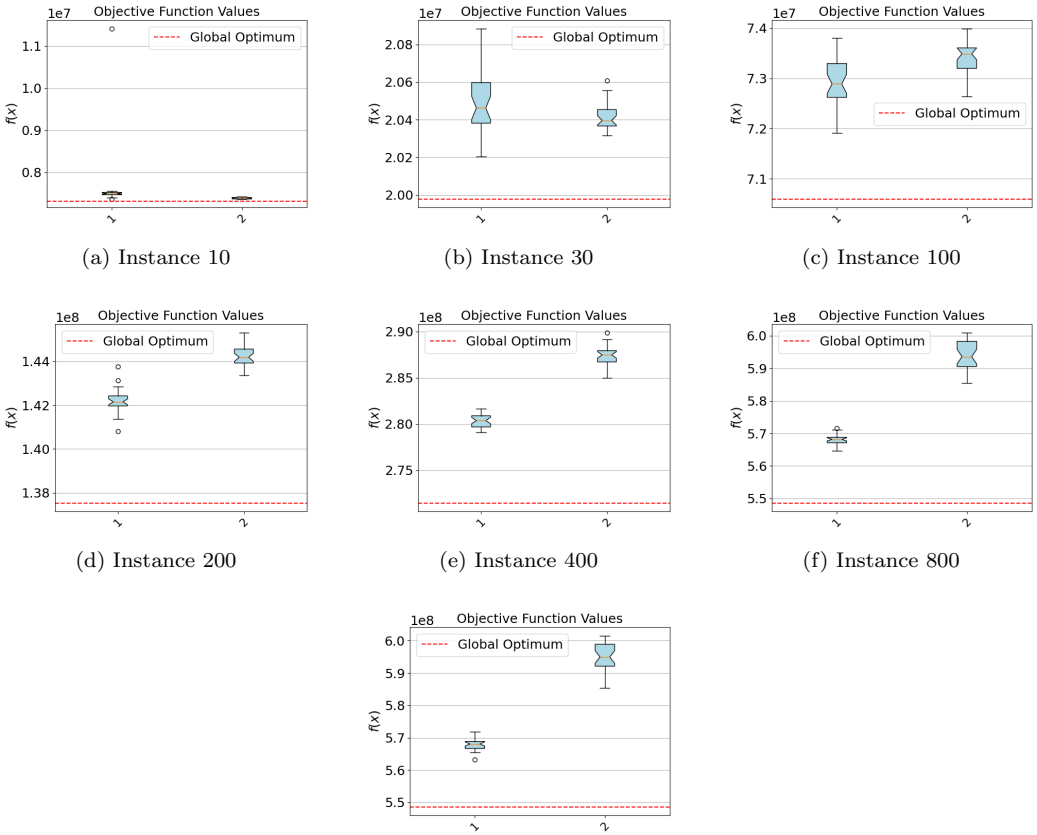
Figure 6 illustrates these distributions visually. The boxplots confirm that for $N \geq 100$, the Proposed VNS not only achieves better median values but also exhibits lower variance, indicating greater robustness.

5.6.3. Convergence Behavior Analysis

Figure 7 provides temporal insights into the search dynamics by showing the distribution of best-so-far objective values at different execution stages (12.5%, 25%, 37.5%, etc.). The plots reveal a fundamental behavioral difference between the two VNS variants.

The Standard VNS (labeled "2", orange line) exhibits a "cliff-like" convergence profile. It descends rapidly in the first 12.5% of execution, then immediately plateaus with extremely tight distributions (small boxes). While this indicates high repeatability across runs, it signals premature convergence: the algorithm becomes trapped in the first local optimum encountered and lacks sufficient diversification mechanisms to escape, effectively stagnating for the remaining 85% of the computational budget.

In contrast, the Proposed VNS (labeled "1", blue boxes) displays substantially wider interquartile ranges during the initial 25% of execution, particularly visible in instances $N \geq 200$. This variance is not a weakness but validates the effectiveness of the proposed diversification operators. By forcing transitions between active and inactive facility configurations, these operators enable the algorithm to traverse multiple basins of attraction, temporarily increasing solution variance while systematically filtering inferior local optima. Critically, as execution progresses beyond 50%, the distributions tighten considerably, demonstrating that the algorithm successfully identifies



(a) Instance 10 (b) Instance 30 (c) Instance 100

(d) Instance 200 (e) Instance 400 (f) Instance 800

(g) Instance 1600

Figure 6: Distribution of objective function values for Proposed VNS (Novel, left boxplot) vs. Standard VNS (Basic, right boxplot) across all instance sizes. The red dashed line represents the optimal solution found by branch-and-bound (Gurobi). The novel approach demonstrates superior scalability.

and converges toward high-quality regions of the solution space. The final distributions (at 100% execution) show medians consistently 4-5% lower than the Standard VNS. Thus, the initial exploration phase, though appearing less stable, constitutes a strategic investment that yields superior solution quality in the long run.

5.7. Ablation Study

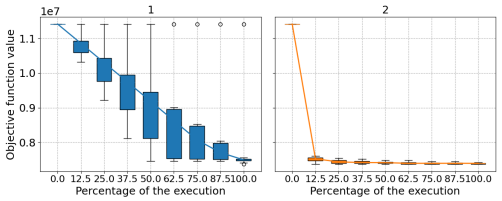
[TO BE WRITTEN] *Ablation over the five variants listed above, attributing the observed performance to individual algorithmic components.*

5.8. Managerial Implications

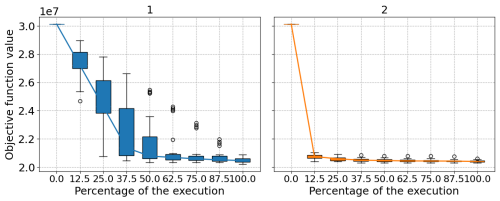
From a practical standpoint, our results suggest a tiered decision framework for supply chain managers in the pistachio industry based on network complexity. Networks with fewer than 100 facilities per echelon should employ commercial solvers (Gurobi, CPLEX) to guarantee optimality within reasonable computational timeframes. Medium-scale networks (100–400 facilities) benefit most from the proposed VNS, which provides near-optimal solutions (approximately 3.3% gap) in minutes rather than hours, offering a compelling balance between solution quality and computational efficiency. Large-scale international supply chains (exceeding 800 facilities) require metaheuristic approaches, as exact methods become computationally prohibitive due to memory constraints and exponential runtime growth. The transparency of our integer-based encoding and decoding algorithm facilitates integration with enterprise resource planning systems, enabling seamless deployment in existing operational infrastructures. Moreover, the solution quality achieved by the proposed VNS remains within acceptable bounds for strategic planning decisions, such as facility location, capacity allocation, and waste management infrastructure design. Implementation requires only standard computational resources, making the approach accessible to organizations without specialized high-performance computing capabilities.

6. Conclusion

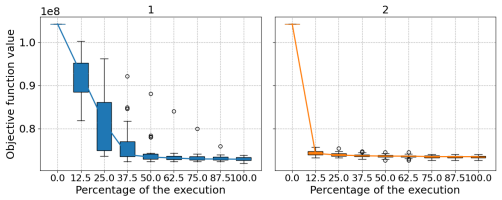
This study presented a comprehensive optimization framework for the pistachio supply chain network, addressing both forward product flows and reverse logistics of by-products. Our primary contributions are twofold: (1) we introduced an integer-based priority encoding with a formally specified backward decoding algorithm, addressing critical methodological gaps in



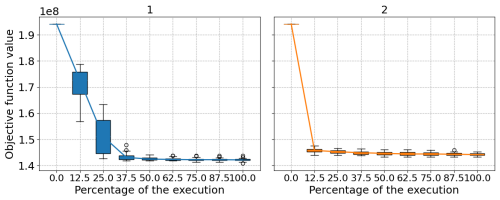
(a) Instance 10



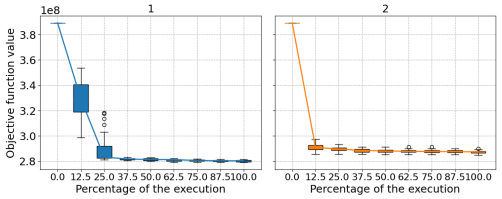
(b) Instance 30



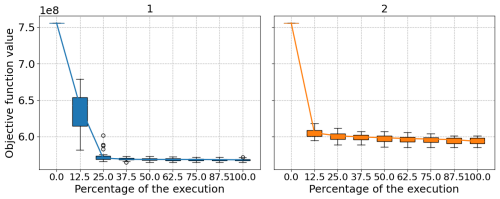
(c) Instance 100



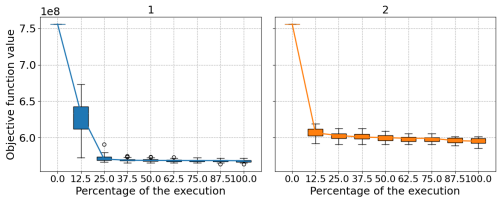
(d) Instance 200



(e) Instance 400



(f) Instance 800



(g) Instance 1600

Figure 7: Convergence curves comparing Proposed VNS (left/blue) vs. Standard VNS (right/orange) across all instance sizes. The x-axis represents the number of objective function evaluations, while the y-axis shows the best solution found up to that point. The Proposed VNS demonstrates faster convergence and better final solutions, particularly for larger instances.

prior work and ensuring full reproducibility; and (2) we developed four novel problem-specific neighborhood operators that exploit the active/inactive node structure inherent to capacity-constrained supply chains, demonstrating statistically significant improvements over standard permutation operators in a comprehensive computational study.

Comprehensive computational experiments benchmarked the proposed VNS against a Genetic Algorithm (GA), Iterated Local Search (ILS), and the exact solver Gurobi. The results reveal a clear performance stratification across instance sizes. For small instances ($N \leq 30$), simple local search methods such as ILS are sufficient and highly efficient, often outperforming more sophisticated approaches. However, as problem complexity increases to medium-sized instances ($N \approx 100$), the proposed VNS becomes statistically superior to both GA and ILS, demonstrating its enhanced search capability. This advantage becomes even more pronounced in large-scale instances ($N \geq 800$), where exact methods become computationally intractable due to memory and time constraints. In these challenging scenarios, the proposed VNS demonstrates remarkable scalability, consistently identifying high-quality feasible solutions that significantly outperform the evolutionary approach, establishing VNS as a promising approach for practical, large-scale supply chain optimization.

Regarding future research, two primary directions emerge. First, further variations of the problem model should be explored, such as incorporating explicit closed-loop feedback loops. Second, and critically, there is a need to investigate new encoding methodologies specifically designed to enable matheuristics—hybrid approaches combining metaheuristics with exact solvers. The priority-based encoding used in this work is highly effective for pure metaheuristics, as it enables efficient local search through the solution space. However, integrating exact methods (such as branch-and-bound) into the VNS framework would require a reverse transformation: converting the decoded transportation tree back into the original MILP variable space. This reverse mapping is extremely challenging because the decoding process involves complex, non-linear decisions (such as the proportional allocation rules in Processing Centers and the greedy priority-based flow assignments) that cannot be trivially inverted. Developing alternative encoding schemes that preserve a bidirectional mapping between heuristic representations and mathematical programming formulations could unlock powerful hybrid decomposition strategies, enabling the VNS to identify promising regions while exact solvers refine solutions with provable bounds. Bridging this gap be-

tween heuristic flexibility and exact solver precision remains a vital frontier for solving such complex, large-scale supply chain problems.

The source code used in this study are openly available at: `https://github.com/raissagd/Pistachio_CLSC`.

References

Ahmadi, S., Amin, S.H., 2019. An integrated chance-constrained stochastic model for a mobile phone closed-loop supply chain network with supplier selection. Journal of Cleaner Production 226, 988–1003.

Aranha, C., Camacho Villalón, C.L., Campelo, F., Dorigo, M., Ruiz, R., Sevaux, M., Sørensen, K., Stützle, T., 2022. Metaphor-based metaheuristics: A call for action. Swarm Intelligence 16, 1–6.

Banasik, M., Stanisławska-Sachadyn, A., Sachadyn, P., 2016. A simple modification of pcr thermal profile applied to evade persisting contamination. Journal of Applied Genetics 57, 409–415.

Bartzas, G., Komnitsas, K., 2017. Life cycle analysis of pistachio production in greece. Science of the Total Environment 595, 13–24.

Campelo, F., Aranha, C., 2023. Lessons from the evolutionary computation bestiary. Artificial Life 29, 421–432.

Casper, R., Sundin, E., 2018. Reverse logistics transportation and packaging concepts in automotive remanufacturing. Procedia Manufacturing 25, 154–160.

Cheraghalipour, A., Roghanian, E., 2022. A bi-level model for a closed-loop agricultural supply chain considering biogas and compost. Environment, Development and Sustainability , 1–47.

Devika, K., Jafarian, A., Nourbakhsh, V., 2014. Designing a sustainable closed-loop supply chain network based on triple bottom line approach: A comparison of metaheuristics hybridization techniques. European Journal of Operational Research 235, 594–615.

Doula, M.K., Kouloumbis, P., Elaiopoulos, K., 2014. Turning wastes into valuable materials: Valorization of pistachio wastes in the agricultural

sector, in: Proceedings of the SYMBIOSIS International Conference, pp. 19–21.

Dronavalli, S., Kang, M.S., 2019. Microgametophytic selection for identifying maize with resistance to aspergillus spp. and aflatoxin. Journal of Crop Improvement 33, 363–371.

Fathollahi-Fard, A.M., Hajiaghaei-Keshteli, M., 2018. A stochastic multi-objective model for a closed-loop supply chain with environmental considerations. Applied Soft Computing 69, 232–249.

Gen, M., Altiparmak, F., Lin, L., 2006. A genetic algorithm for two-stage transportation problem using priority-based encoding. OR Spectrum 28, 337–354.

Gendreau, M., Potvin, J.Y. (Eds.), 2010. Handbook of Metaheuristics. 2 ed., Springer.

Hansen, P., Mladenović, N., Moreno Pérez, J.A., 2010. Variable neighbourhood search: Methods and applications. Annals of Operations Research 175, 367–407.

Hokmabadi, H., 2018. Pistachio wastes in iran and the potential to recapture them in value chain. Pistachio and Health Journal 1, 1–12.

Hussain, A., Riaz, S., Amjad, M.S., Haq, E.u., 2022. Genetic algorithm with a new round-robin based tournament selection: Statistical properties analysis. Plos one 17, e0274456.

Kirkpatrick, S., Gelatt Jr, C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. science 220, 671–680.

Pishvaee, M.S., Farahani, R.Z., Dullaert, W., 2010. A memetic algorithm for bi-objective integrated forward/reverse logistics network design. Computers & Operations Research 37, 1100–1112.

Rajabi-Kafshgar, A., Gholian-Jouybari, F., Seyedi, I., Hajiaghaei-Keshteli, M., 2023. Utilizing hybrid metaheuristic approach to design an agricultural closed-loop supply chain network. Expert Systems with Applications 217, 119504.

Statista, 2023. Leading producers of pistachios worldwide in 2022 and 2023. https://www.statista.com/statistics/933042/global-pistachio-production-by-country/.

Tinós, R., Przewozniczek, M.W., Whitley, D., Chicano, F., 2024. Iterated local search with linkage learning. ACM Transactions on Evolutionary Learning and Optimization 4, 1–29.

Xie, Y., Sheng, Y., Qiu, M., Gui, F., 2022. An adaptive decoding biased random key genetic algorithm for cloud workflow scheduling. Engineering Applications of Artificial Intelligence 112, 104879.

Zohal, M., Soleimani, H., 2016. Developing an ant colony approach for green closed-loop supply chain network design: A case study in the gold industry. Journal of Cleaner Production 133, 314–337.